Prowadzący zajęcia: Artur Samojluk

Technologie i prompty AI

Od Road Map do lokalnie działającej wersji v1. Wybór narzędzi i kodowanie z pomocą czatu AI.

Cel ćwiczenia

Dobierzecie technologie do projektu z laba 02 i uruchomicie pierwszą wersję aplikacji. Przełożycie szkice, strukturę danych i funkcje MVP na kolejne polecenia dla AI, a następnie sprawdzicie wygenerowany kod oraz działanie v1.

1. Uruchomcie wybrany bezpłatny czat AI

Każda osoba otwiera jeden wybrany serwis, zakłada konto lub loguje się na istniejące i rozpoczyna nową rozmowę dotyczącą projektu. Możecie korzystać z różnych narzędzi. Wybierzcie dostępny wariant bezpłatny i zanotujcie nazwę używanego czatu.

Czat AI	Adres i wariant do ćwiczenia
ChatGPT	chatgpt.com - plan Free [1]
Claude (Anthropic)	claude.ai - plan Free [2]
Gemini (Google)	gemini.google.com - dostęp bez płatnego planu AI [3]

Bezpłatne warianty mają limity i zmienną dostępność funkcji. Ćwiczenie wykorzystuje zwykłą rozmowę w przeglądarce oraz lokalne uruchamianie kodu; nie wymaga kluczy API ani płatnej subskrypcji. Claude Code jest osobnym narzędziem względem czatu Claude. Źródła i stan dostępności: 27.10.2026, strona 4.

Na początek wyślijcie: „Wyjaśnij na przykładzie rezerwacji sali, co robi widok, serwer i baza danych”. Omówcie odpowiedź w zespole. Korzystajcie z fikcyjnych danych; pomijajcie hasła i klucze dostępu.

2. Dobierzcie technologie i opiszcie ich role

Wybierzcie zestaw dla własnego projektu i uzasadnijcie go w 5-8 zdaniach. Poniżej prosty wariant dla SalaSpot; możecie wybrać inny, dopasowany do swoich umiejętności.

Obszar	Przykładowy wybór	Rola w projekcie
Widoki	HTML, CSS, JavaScript	Treść, wygląd i obsługa formularzy.
Serwer	Python 3 + Flask	Żądania, walidacja i reguły działania.
Dane	SQLite, moduł sqlite3	Trwały zapis w lokalnym pliku bazy.
Praca zespołu	Edytor kodu, Git, GitHub	Edycja plików i historia zmian.

Zapiszcie wybór i wersje narzędzi w docs/technologie.md. Przykłady promptów na kolejnych stronach zakładają powyższy wariant.

Kontekst i start projektu

Prompt to polecenie dla AI. Powinien określać cel, kontekst, zakres zmiany i sposób sprawdzenia wyniku. Dopasujcie przykłady do własnej mapy v1.

3. Przekażcie mapę projektu i wybierzcie rozwiązanie

P1. Wybór technologii - szablon do uzupełnienia

Pracujemy we dwoje nad projektem [tytuł]. Cel v1: [cel]. Widoki: [lista W01...]. Dane: [tabele i relacje]. Funkcje MVP: [F01...]. Nasz system operacyjny i umiejętności: [opis]. Zaproponuj dwa proste zestawy technologii do uruchomienia lokalnie bez płatnych usług. Wyjaśnij role składników, wybierz prostszy wariant i uzasadnij go. Zwróć też listę potrzebnych narzędzi. Na tym etapie przygotuj plan.

Wklejcie treść docs/roadmap-v1.md oraz opisy szkiców. Jeśli czat przyjmuje pliki, możecie je dołączyć. Oceńcie propozycję i zapiszcie własną decyzję. Przy zmianie technologii dostosujcie również następne prompty i polecenia uruchomienia.

4. Przygotujcie i uruchomcie szkielet aplikacji

P2. Minimalny szkielet - przykład SalaSpot

Przygotuj szkielet lokalnego SalaSpot v1: HTML/CSS, Python 3, Flask i docelowo SQLite przez sqlite3. Plik startowy: app.py. Widoki: W01 lista sal, W02 formularz rezerwacji, W03 moje rezerwacje. Na razie użyj danych przykładowych w kodzie. Zwróć strukturę katalogów, pełną zawartość plików, requirements.txt i instrukcję startu. Dodaj .gitignore dla .venv i lokalnej bazy. Podaj prosty sposób sprawdzenia każdego widoku. Zachowaj zakres v1.

Przykładowe polecenia uruchomienia - Python + Flask

Poniższe polecenia dotyczą przykładowego wariantu Python + Flask. Po przygotowaniu plików app.py i requirements.txt otwórzcie terminal w ich katalogu. Wykonajcie tylko jeden blok, odpowiedni dla swojego systemu [4-5]. Przy wyborze innej technologii zastosujcie właściwe dla niej polecenia.

```
Windows (PowerShell):
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m flask --app app run

macOS / Linux:
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m flask --app app run
```

Zainstalujcie Python 3, jeśli nie jest dostępny. Otwórzcie http://127.0.0.1:5000. Adres działa na komputerze, na którym uruchomiono serwer. Zatrzymacie go skrótem Ctrl+C. Każda osoba powinna umieć uruchomić projekt na swoim komputerze.

Zasada pracy po każdym kroku

Przeczytajcie kod, zapiszcie zmianę, uruchomcie aplikację i sprawdźcie efekt. Po udanym sprawdzeniu wykonajcie commit. Kolejne prompty wysyłajcie z aktualnymi plikami lub ich istotnymi fragmentami.

Widoki, baza i funkcje v1

Realizujcie cykl z laba 02: widoki, struktura bazy, funkcjonalności. W przykładzie SalaSpot użyjcie lokalnego użytkownika demonstracyjnego nr 1 i opiszcie ten tryb w README.md.

5. Zamieńcie plan na małe zadania dla AI

P3. Widoki zgodne ze szkicami

Dopasuj istniejące widoki SalaSpot do szkiców: [opisy W01, W02, W03]. W01 pokazuje sale, W02 zawiera wybór sali oraz początek i koniec rezerwacji, W03 listę moich rezerwacji. Dodaj nawigację, etykiety pól i miejsca na komunikaty sukcesu oraz błędów. Zachowaj obecną technologię. Podaj ścieżki i pełny kod zmienianych plików oraz sposób sprawdzenia widoków. Aktualne pliki: [wklej kod].

P4. SQLite zamiast danych tymczasowych

Dodaj SQLite przez sqlite3 zgodnie z mapą: uzytkownik(id, imie, email), sala(id, nazwa), rezerwacja(id, uzytkownik_id, sala_id, poczatek, koniec, status). Oznacz PK i FK oraz włącz kontrolę kluczy obcych. Używaj zapytań parametryzowanych. Dodaj osobne polecenie inicjalizacji bazy, fikcyjne sale i użytkownika demo nr 1. Ponowny start aplikacji ma zachowywać dane. Zwróć zmienione pliki, polecenie inicjalizacji i sposób sprawdzenia zapisu. Aktualne pliki: [wklej kod].

P5. Podstawowe funkcjonalności

Na obecnym kodzie wykonaj kolejno F01 sprawdzenie dostępności sal w wybranym terminie, F02 utworzenie rezerwacji i F03 anulowanie własnej rezerwacji użytkownika demo nr 1. Sprawdzaj dane na serwerze: wymagane pola, początek przed końcem i brak kolizji aktywnych rezerwacji tej sali. Anulowanie zmienia status; anulowane rezerwacje nie blokują terminu. Po każdej funkcji podaj zmienione pliki i kroki sprawdzenia. Zacznij od F01. Aktualne pliki: [wklej kod].

Przykład prostej reguły: terminy nakładają się, gdy oba poniższe warunki są spełnione. Sprawdzajcie wyłącznie tę samą salę i aktywne rezerwacje.

```
nowy_poczatek < istniejacy_koniec
AND nowy_koniec > istniejacy_poczatek
```

Rezerwacja 10:00-11:00 blokuje 10:30-11:30, ale pozwala zarezerwować 11:00-12:00. Porównywane wartości muszą zawierać datę i godzinę w spójnym formacie.

Własny projekt

Wykonajcie wszystkie 3-5 funkcji MVP wybrane w labie 02. Po F01 sprawdźcie wynik, potem poproście o F02 i kolejne funkcje. Jeśli Wasz zakres v1 obejmuje logowanie, uwzględnijcie je w implementacji i testach.

Poprawki i odbiór wersji v1

Wynik rozmowy z AI wymaga uruchomienia i sprawdzenia. Zapisujcie różnicę między oczekiwanym a rzeczywistym działaniem.

6. Zgłaszajcie błędy z konkretnymi dowodami

P6. Naprawa błędu - szablon

Uruchomiłem projekt poleceniem [polecenie] w [system i wersje]. Kroki odtworzenia: [1, 2, 3]. Oczekiwany wynik: [opis]. Rzeczywisty wynik: [opis]. Komunikat błędu: [pełny komunikat]. Aktualny kod: [istotne pliki]. Wskaż przyczynę, popraw tylko potrzebne fragmenty i wyjaśnij zmianę. Podaj sposób sprawdzenia poprawki oraz tego, czy wcześniejsze funkcje nadal działają.

7. Sprawdźcie v1 względem mapy projektu

P7. Plan sprawdzenia wersji v1

Porównaj kod [pliki] z mapą v1 [treść]. Wskaż braki i przygotuj scenariusze sprawdzenia dla każdej funkcji: dane wejściowe, kroki i oczekiwany wynik. Uwzględnij błędy i zachowanie danych po restarcie. Oddziel ocenę kodu od testów, które trzeba wykonać w działającej aplikacji.

Przykład testu SalaSpot	Oczekiwany wynik
Rezerwacja wolnej sali: 10:00-11:00	Rekord w bazie i na liście W03.
Ta sama sala: 10:30-11:30	Odrzucenie konfliktu i komunikat.
Ta sama sala: 11:00-12:00	Przyjęcie terminu stykającego się z poprzednim.
Puste pole lub koniec przed początkiem	Komunikat i brak niepoprawnego zapisu.
Anulowanie własnej rezerwacji	Zmiana statusu i zwolnienie terminu.
Zatrzymanie i ponowny start serwera	Zachowane zapisane rezerwacje.

Każdy scenariusz wykonajcie samodzielnie. Wyniki oznaczcie: działa / błąd / niewykonany. Przy błędzie zapiszcie kroki odtworzenia i poprawkę.

8. Zapiszcie rezultaty i oddajcie projekt

- ☐ Kod v1 realizuje 3-5 funkcji MVP, korzysta z trwałej bazy i uruchamia się u obu osób.
- ☐ README.md opisuje start i inicjalizację bazy; docs/technologie.md uzasadnia wybrane narzędzia.
- ☐ docs/prompty-v1.md zawiera ważne prompty, decyzje zespołu i opis własnych poprawek.
- ☐ docs/testy-v1.md zawiera rzeczywiste wyniki; GitHub pokazuje wkład obu osób, a LaboChat.com - stan zadań.

Sposób oddania i termin

Przekażcie link do repozytorium GitHub i przestrzeni projektu w LaboChat.com. Termin: do kolejnych zajęć. Obie osoby powinny umieć uruchomić aplikację i wyjaśnić działanie wybranej funkcji od widoku do zapisu danych.

Oficjalne materiały, sprawdzone 27.10.2026: [1] ChatGPT - plany; [2] Claude - plany; [3] Gemini - dostęp i limity; [4] Flask - instalacja; [5] Flask - pierwsze uruchomienie.